

2026-2 INHATC

피지컬 AI 응용

4주차 · 비지도학습과 모델 선택

김태령

PART 1 · 비지도학습이란

정답이 없는데 무엇을 배우나, 잘했는지 어떻게 아나

PART 2 · 군집

K-평균 · 정답을 숨기고 돌려 보기 · K는 누가 정하나

PART 3 · 차원축소

PCA · 특성 5개를 2개로 눌러 그림으로 보기

PART 4 · 모델 선택

하이퍼파라미터 · 과적합 곡선 · 교차검증

정리 · 과제

핵심 3가지와 5주차 딥러닝 예고

오늘의 목표 - 정답이 없어도 데이터에서 구조를 찾을 수 있고, 그 결과를 사람이 판단해야 한다는 것을 손으로 확인한다.



지난주에 외운 것 · 그리고 남긴 질문

외운 것 - 5단계

회귀와 분류는 두 군데만 다르다

```
X_train, X_test, y_train, y_test =  
train_test_split(X, y, ...)
```

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

```
accuracy_score(y_test, pred)
```

질문

- 로지스틱 회귀 0.875 > 결정 트리 0.775
- 가장 단순한 모델이 1등
- 복잡한 모델이 항상 좋은 게 아니라면, 그러면 어떻게 고르나?

PART 1

비지도학습이란 무엇인가

Unsupervised Learning

현실 문제

“공장 로봇 200대에서 센서 로그 10만 건이 쌓였습니다. 고장 예측 모델을 만드세요.”

- 지난주 방식이라면 - 10만 건마다 ‘정상 / 고장’ 정답을 붙인 뒤 `LogisticRegression().fit(X, y)`.
- 그런데 그 정답은 누가 답니까? 한 건에 10초씩 잡아도 사람 한 명이 280시간
- 실제 현장에서는 정답이 없는 데이터가 정답이 있는 데이터보다 백 배쯤 많음
- **정답이 없어서가 아니라, 정답을 붙이는 게 너무 비싸다.**

정답 없이 무엇을 배우나

비슷한 것끼리 묶는다

10만 건이 사실은 서너 가지 상황의 반복이라면? 그 서너 덩어리를 찾는다.

군집

눌러서 그려 본다

센서가 20개면 그림으로 못 그린다. 정보를 최대한 남기고 2개로 줄여 눈으로 본

·
차원축소

튀는 것을 찾는다

대부분과 다른 소수를 찾는다. 오늘은 다루지 않지만 원리는 같다.

이상 탐지

그런데 정답이 없으면

정확도는 불가, 대체할 방법이 필요

지도학습과 나란히 놓고 보기

지도학습 (지난주)

정답 y 가 있다

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

```
accuracy_score(y_test, pred)
```

- 잘했는지: 정답과 비교하면 끝
- 결과: 새 데이터의 답

비지도학습 (오늘)

x 만 있다

```
model.fit(X)
```

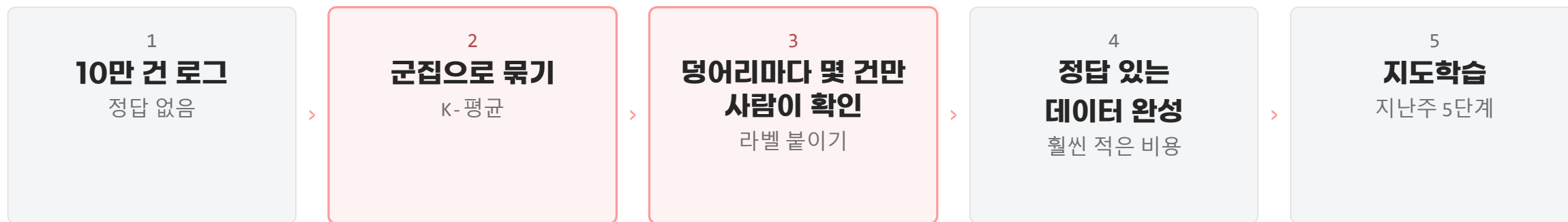
```
labels = model.labels_ 또는 Z = model.transform(X)
```

```
inertia_ · silhouette_score · 사람의 눈
```

- 잘했는지: 정답이 없으니 ‘덩어리가 얼마나 뚜렷한가’ 로 대신 측정
- 결과: 데이터의 구조 (묶음, 축)

y 가 빠지고, 마지막 동사가 predict 대신 labels_ 나 transform 으로

실무에서 비지도학습이 쓰이는 방식



- 비지도학습은 정답을 싸게 만드는 도구
- 10만 건에 정답을 다는 대신, 덩어리 4개에서 20건씩 80건만 확인

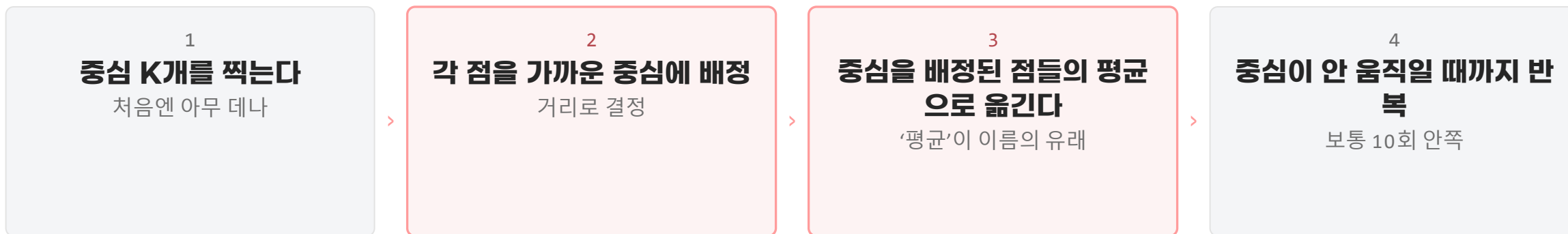
PART 2

군집 - 비슷한 것끼리 묶기

Clustering with K-Means

K-평균의 작동 · 스케일 · 정답을 숨기고 돌려 보기 · K는 누가 정하나

K-평균이 하는 일 - 네 단계의 반복



- 필요한 것은 단 하나 - 거리. 그래서 스케일이 다르면 큰 숫자 쪽으로 끌려감.
- K(몇 덩어리로 묶을지)는 사람이
- 처음 찍는 위치에 따라 결과가 달라질 수 있어 `random_state` 를 주고, `sklearn` 은 기본으로 여러 번 돌려 가장 좋은 것을 고름(`n_init`).

코드 - 골격은 그대로, y 만 없다

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X = clf[["온도", "진동"]].values          # y 는 쓰지 않는다 (숨긴다)
Xs = StandardScaler().fit_transform(X)    # ① 스케일 맞추고
km = KMeans(n_clusters=2, random_state=42) # ② 만들고 (K 는 사람이)
km.fit(Xs)                                # ③ 학습하고 - y 없음
labels = km.labels_                       # ④ 각 점의 군집 번호 (0 또는 1)
print(km.inertia_)                        # ⑤ 평가 - 정확도 대신 관성
```

관성(inertia) - 각 점에서 자기 중심까지 거리 제곱의 합. 단, K 를 늘리면 무조건 작아진다

스케일을 먼저 맞추는 이유

스케일 없이

온도 15~45, 진동 0~5

- 온도 차이 10도 = 거리 10
- 진동 차이 2 = 거리 2
- **거리가 온도로만 결정된다**
- 진동은 있으나 마나 한 특성이 된다

```
KMeans(...).fit(X)
```

표준화 후

둘 다 평균 0, 표준편차 1

- 온도와 진동이 같은 무게로 거리에 들어간다
- ```
StandardScaler().fit_transform(X)
```
- **거리 기반 모델(KMeans · KNN · PCA)은 언제나 이것부터**
  - 트리 계열만 예외 (지난주 내용)

```
KMeans(...).fit(Xs)
```

## 실습 ① 정답을 숨기고 돌려 보기

```
지난주와 같은 고장 데이터 (온도·진동 → 고장). 이번엔 y 를 안 준다.
Xs = StandardScaler().fit_transform(clf[["온도", "진동"]].values)
km = KMeans(n_clusters=2, random_state=42).fit(Xs)

군집 번호와 진짜 정답을 교차표로 비교
print(pd.crosstab(km.labels_, clf["고장"],
 rownames=["군집"], colnames=["실제"]))
```

|      | 실제 0 | 실제 1 |
|------|------|------|
| 군집 0 | 25   | 79   |
| 군집 1 | 68   | 28   |

**일치율 73.5%**

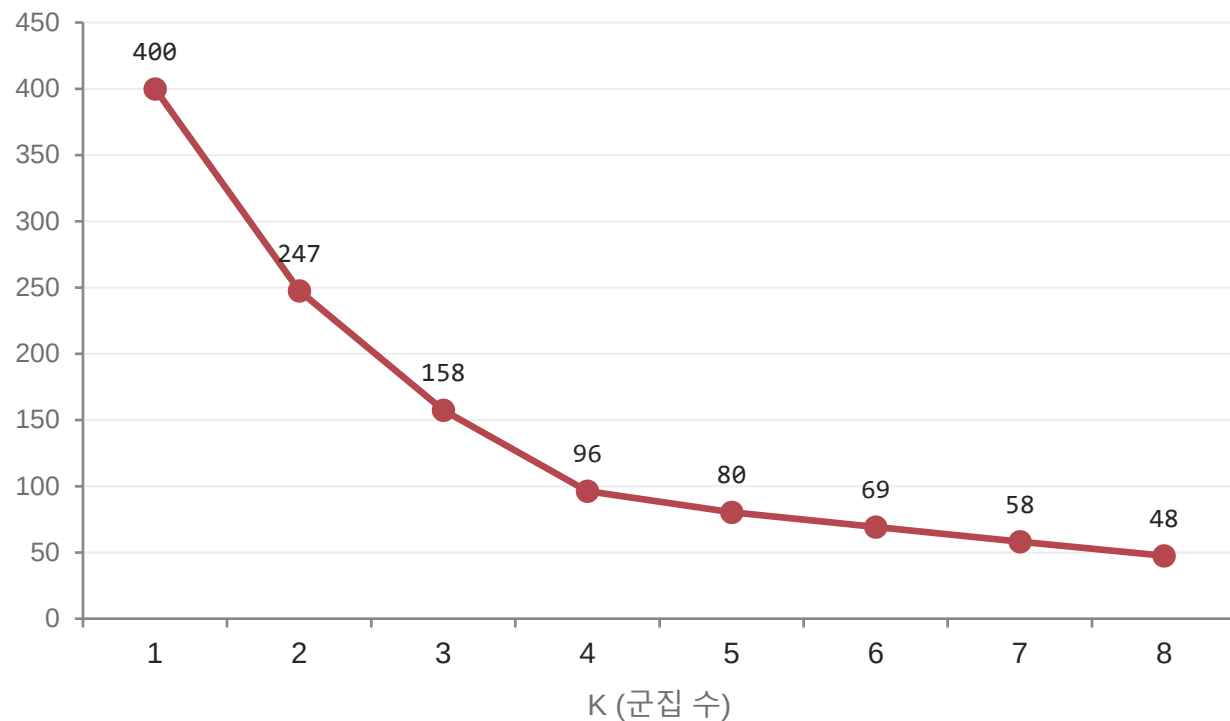
주의 - 군집 번호 0·1 은 알고리즘이 임의로 붙인 것이라 '0 = 고장'이 아님

## 못 가른 27%

- KMeans 가 찾은 두 중심(원래 단위): 군집 0 = (온도 30.9, 진동 3.5) · 군집 1 = (온도 28.6, 진동 1.1).
- 두 중심은 온도가 거의 같고 진동만 다름. 즉 KMeans 는 ‘진동이 높은 쪽 / 낮은 쪽’ 으로 가르게 됨
- 그런데 진짜 고장 규칙은  $0.25 \times \text{온도} + 1.2 \times \text{진동} > 10$  - 온도도 한몫합니다. 온도가 높고 진동이 낮은 고장은 군집 1로 잘못 들어갑니다.
- 비지도학습은 ‘데이터가 뭉쳐 있는 모양’을 찾을 뿐, ‘우리가 알고 싶은 정답’을 찾는 게 아님
- 둘이 겹칠 때 잘 맞고, 어긋나면 틀림

그래서 실무 절차는 ‘군집 → 사람이 확인 → 라벨 → 지도학습’

## K 는 누가 정하나 ① 엘보우 방법



- K 를 늘리면 관성은 무조건 줄어듦. K = 200이면 0입니다. 그래서 '가장 작은 K' 는 답이 아님
- 대신 그래프가 팔꿈치처럼 꺾이는 곳을 찾으면 되지만 그 뒤로는 K 를 늘려도 별로 안 좋아짐
- 이 그래프에서는 꺾이는 곳이 뚜렷하지 않습니다. 3? 4? - 데이터가 명확한 덩어리로 뭉음쳐 있지 않기 때문
- 이런 일은 실제 데이터에서 아주 흔함

## K 는 누가 정하나 ② 실루엣 점수

| K | 실루엣 점수       |                     |
|---|--------------|---------------------|
| 2 | <b>0.355</b> | 우리가 원하는 값 (정상 / 고장) |
| 3 | 0.382        |                     |
| 4 | <b>0.418</b> | 데이터가 가장 뚜렷하게 갈리는 값  |
| 5 | 0.411        |                     |
| 6 | 0.355        |                     |
| 7 | 0.370        |                     |
| 8 | 0.382        |                     |

- 실루엣 점수 - 각 점이 자기 덩어리에 얼마나 잘 붙어 있고 남의 덩어리와 얼마나 떨어져 있는가. 1에 가까울수록 좋음.
- 데이터는  $k = 4$  가 가장 뚜렷하다고 말함, 그런데 우리는 '정상 / 고장' 두 가지를 알고 싶음
- 목적이  $k$  를 정함. 데이터는 참고 의견을 낼 뿐
- 이것이 '정답이 없다'는 말의 실제 의미. 평가 지표가 없는 게 아니라, 지표가 최종 결정을 내려 주지 않음

## 실습 ② K 를 바꿔 보기

```
from sklearn.metrics import silhouette_score

for k in range(2, 7):
 km = KMeans(n_clusters=k, random_state=42).fit(Xs)
 sil = silhouette_score(Xs, km.labels_)
 print(k, round(km.inertia_, 1), round(sil, 3))
```

### 확인 1

관성이 K에 따라 어떻게 변하나? 한 번이라도 커지는 경우가 있나?

### 확인 2

실루엣이 가장 큰 K 는? 그 K 로 묶으면 각 군집에 고장이 몇 %인가? (crosstab)

### 확인 3

여러분이라면 K를 몇으로?

## 군집에서 자주 하는 실수

### ① 스케일을 안 맞춘다

거리 기반이므로 큰 숫자 특성이 결과를 독점한다.

`StandardScaler` 먼저

### ② 군집 번호를 정답으로 착각

0~1 은 임의의 번호. 어느 군집이 무엇인지는 사람이 붙인다.

`crosstab` 으로 확인

### ③ K 에 정답이 있다고 믿는다

엘로우·실루엣은 참고. 최종 결정은 목적이 한다.

목적 → K

### ④ `random_state` 를 안 준다

시작점이 달라 돌릴 때마다 번호가 바뀐다.

`random_state=42`

### ⑤ 군집 = 정답이라 믿고 배포

군집은 ‘뭉친 모양’ 일 뿐. 반드시 사람이 한 뒤 라벨로 쓴다.

군집 → 확인 → 라벨

### ⑥ `train_test_split` 을 찾는다

정답이 없으니 시험이 없다. 없는 게 맞다.

`fit(X)` 한 번

②와 ⑤가 실무에서 사고 남. 군집 결과를 곧바로 ‘고장 예측 모델’이라고 부르는 순간 문제가 시작

PART 3

# 차원 축소 - 눌러서 그려 보기

Dimensionality Reduction with PCA

왜 줄이나 · PCA 한 문장 · 센서 5개를 2개로 · 12주차 임베딩 예고

## 왜 줄이나 - 특성이 5개면 그림을 못 그린다

### 특성 2개 (지난주)

온도 · 진동

- x축 온도, y축 진동 - 산점도 한 장이면 끝
- 고장이 어디 몰려 있는지 눈에 보인다
- 경계가 직선인지 곡선인지도 보인다

### 특성 5개 (오늘)

온도 · 진동 · 전류 · 소음 · 배터리

- 축이 5개 - 종이에 못 그린다
- 실제 로봇은 센서가 20~50개
- 12주차 이미지 임베딩은 512개
- 정보를 최대한 남기고 2개로 눌러야 눈으로 볼 수 있다

# PCA 한 문장 - 정보가 가장 많이 남는 방향으로 눌러 찍는다

- 손 그림자 놀이를 생각하세요. 손은 3차원인데 벽의 그림자는 2차원. 빛을 어디서 비추느냐에 따라 그림자가 손처럼 보이기도, 막대기처럼 보이기도 함
- PCA 는 '그림자가 원래 모양을 가장 잘 보여 주는 각도' 를 찾아 주는 도구. 그 각도가 주성분(principal component)
- '잘 보여 준다'의 기준 - 눌러도 점들이 최대한 넓게 퍼져 있을 것(분산이 클 것). 뭉개져서 한 점이 되면 정보를 잃은 것

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
Z = pca.fit_transform(Xs) # 결과는 예측이 아니라 새 X → transform
print(pca.explained_variance_ratio_) # 성분마다 원래 정보의 몇 %를 남겼나
```

## 실습 ③ 센서 5개 → 2개

```
센서 5개짜리 표 sensor5 (온도·진동·전류·소음·배터리)
X5 = StandardScaler().fit_transform(sensor5.values)
pca = PCA(n_components=2)
Z = pca.fit_transform(X5)

print(Z.shape) # (200, 2)
print(pca.explained_variance_ratio_) # [0.54 0.23]

plt.scatter(Z[:, 0], Z[:, 1], c=c1f["고장"])
정답 색으로 칠해 보기
```

**54%**

성분 1

**23%**

성분 2

**77%**

둘이 합쳐서 - 5개를 2개로 눌렀는데 3/4가 남음

산점도에서 확인할 것 - 정답 색(고장/정상)이 왼쪽·오른쪽으로 갈리는가? 갈린다면 5개 센서의 정보가 2차원에도 살아 있는 것

## PCA 가 덤으로 알려준 것

```
print(pca.components_[0].round(2))
[0.23 0.57 0.57 0.55 0.06]
온도 진동 전류 소음 배터리
```

- 성분 1 은 진동·전류·소음을 거의 같은 무게( $0.57 \cdot 0.57 \cdot 0.55$ )로 섞은 축. 세 센서가 사실상 한 가지를 재고 있었다는 뜻
- 배터리는 0.06 - 성분 1과 거의 무관. 배터리는 고장과 상관없이 따로 노는 특성
- 우리는 이 데이터를 만들 때 전류와 소음을 진동에서 파생함. PCA 가 그 사실을 데이터만 보고 알아냄.
- 회귀가 '숨긴 규칙'을 찾아냈듯, PCA 는 '숨긴 구조'를 찾아냄

## 12주차 예고 - 같은 코드, 512차원



- (이미지처리시)사진 수백 장을 CLIP 에 넣어 512차원 벡터로 만들고, PCA 두 줄로 2차원에 뿌릴 수 있음
- 그 위에 오늘의 KMeans 를 돌리면, 정답 없이도 '비슷한 사진끼리' 묶임. 이미지 군집
- 특성 5개에서 512개로 늘어날 뿐, 코드는 그대로

PART 4

# 모델 선택과 하이퍼파라미터

Model Selection & Hyperparameters

지난주의 질문 · 하이퍼파라미터 · 과적합 곡선 · 교차검증 · GridSearch · 시험 데이터는 한 번만

## 질문 - 왜 가장 단순한 모델이 이겼나

**0.875**

LogisticRegression

**0.850**

KNeighbors

**0.850**

RandomForest

**0.775**

DecisionTree

- 답 ① 데이터의 경계가 직선이기 때문. 직선을 긋는 모델(로지스틱)이 딱 맞았고, 구불구불 그릴 수 있는 모델은 잡음까지 따라 그림
- 답 ② 결정 트리에 '깊이 제한'을 안 줬음. 제한이 없는 트리는 훈련 데이터를 통째로 외움(훈련 정확도 1.0).
- 즉 문제는 트리가 나쁜 게 아니라, 트리의 손잡이를 안 만진 것. 하이퍼파라미터가 중요

# 파라미터와 하이퍼파라미터

## 파라미터

모델이 데이터에서 스스로 배운다

`model.coef_` · `model.intercept_`

- 선형 회귀의 기울기.절편
- `fit()` 이 정한다. 사람이 손대지 않는다
- 지난주 `[12.09, -142.55]` 가 이것

## 하이퍼파라미터

사람이 `fit` 전에 정해 주는 손잡이

`DecisionTreeClassifier(max_depth=3)`

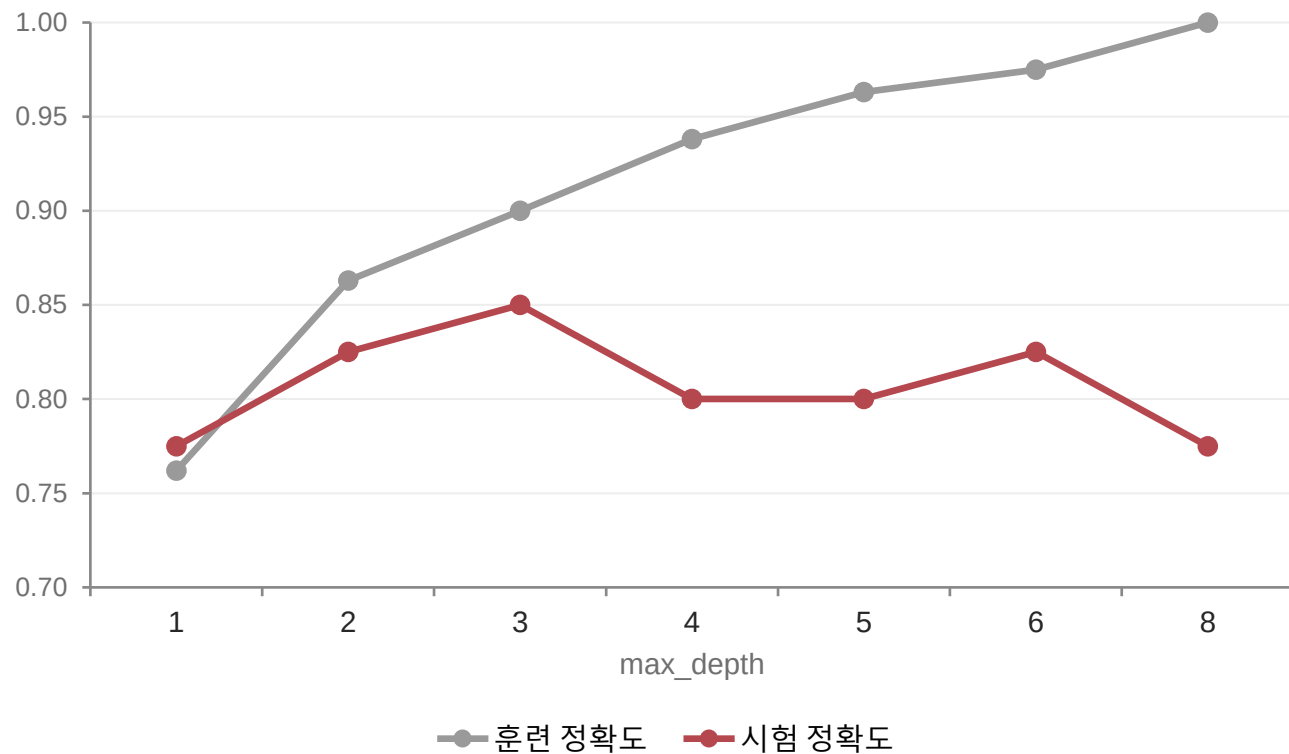
`KMeans(n_clusters=2)` ·

`KNeighborsClassifier(n_neighbors=5)`

- 괄호 안에 쓰는 것은 전부 하이퍼파라미터
- `fit()` 이 정해 주지 않는다. 그래서 우리가 골라야 한다

`KMeans` 의 `n_clusters`. `K` 를 사람이 정해야 했던 그 문제가 바로 하이퍼파라미터 선택

## 실습 ④ 손잡이를 돌려 보기 - max\_depth 1 → 8



```
for d in [1,2,3,4,5,6,8]:
 t = DecisionTreeClassifier(
 max_depth=d, random_state=42)
 t.fit(X_train, y_train)
 print(d, t.score(X_train, y_train),
 t.score(X_test, y_test))
```

- 훈련 점수는 깊이를 늘릴수록 1.0 까지 올라감
- 시험 점수는 3에서 최고(0.85), 그 뒤로 떨어짐
- 둘이 벌어지는 지점부터가 과적합

## 이 곡선을 읽는 법

### 왼쪽 · 깊이 1~2

훈련도 시험도 낮다. 모델이 너무 단순해  
규칙을 못 담는다.

과소적합

### 가운데 · 깊이 3

시험 점수 최고. 규칙은 담고 잡음은 안 담  
았다.

적당함 ← 여기를 고른다

### 오른쪽 · 깊이 5 이상

훈련은 1.0, 시험은 하락. 잡음까지 외웠  
다.

과적합

- 지난주 트리는 깊이 제한이 없어 맨 오른쪽에 있게 됨. `max_depth=3` 만 줘어도 0.775 → 0.85 로, 로지스틱 회귀와 비슷해  
짐.
- 모든 하이퍼파라미터가 이 곡선을 만듦. KNN 의 `n_neighbors`, 신경망의 층 수와 학습 횟수도 같음
- 그래서 모델 선택이란 ‘가장 복잡한 것’도 ‘가장 단순한 것’도 아닌, 이 곡선의 꼭대기를 찾는 일

## 그런데 - 한 번 나눈 점수를 믿을 수 있나

- 위 곡선은 `random_state=42` 로 한 번 나눈 결과. 다르게 나누면 꼭대기가 3이 아니라 4일 수도 있음.
- 시험 데이터가 40개뿐이라 한 개 차이가 2.5%p = 운에 흔들림
- 지난주 배운 교차검증 - 5번 나눠 5번 채점하고 평균 - 이 여기서 필요

| 모델                         | 5번의 점수                                | 평균           |
|----------------------------|---------------------------------------|--------------|
| LogisticRegression         | 0.925 · 0.825 · 0.975 · 0.900 · 0.900 | <b>0.905</b> |
| DecisionTree (max_depth=3) | 0.725 · 0.775 · 0.825 · 0.850 · 0.850 | <b>0.805</b> |
| DecisionTree (제한 없음)       | 0.800 · 0.800 · 0.875 · 0.850 · 0.875 | <b>0.840</b> |

흥미로운 점 - 한 번 나눴을 때는 `depth=3` 이 제한 없는 트리보다 좋았는데, 5번 평균에서는 뒤집힌 한 번의 점수가 얼마나 흔들리는지 보여 주는 실물. 로지스틱 회귀는 어떻게 나눠도 1등임.

## 손잡이를 자동으로 돌려 주는 한 줄 - GridSearchCV

```
from sklearn.model_selection import GridSearchCV

후보 = {"max_depth": [1, 2, 3, 4, 5, 6, 8, 10]}
gs = GridSearchCV(DecisionTreeClassifier(random_state=42), 후보, cv=5)
gs.fit(X_train, y_train) # 후보 8개 × 5겹 = 40번 학습

print(gs.best_params_) # {'max_depth': 4}
print(round(gs.best_score_, 3)) # 0.831 (교차검증 평균)
print(round(gs.score(X_test, y_test), 3)) # 0.800 (시험 데이터 - 딱 한 번)
```

- ‘후보를 전부 돌려서 교차검증 평균이 가장 높은 것을 고른다’ - 우리가 손으로 한 일을 한 줄로
- 후보가 두 개 이상이면 조합이 곱해짐. max\_depth 8개 × min\_samples\_leaf 5개 = 40조합 × 5겹 = 200번.

# 시험 데이터를 여러 번 보면 안 되는 이유

## 잘못된 절차

시험 데이터로 손잡이를 고른다

- depth 1~8 을 시험 데이터로 채점해 가장 좋은 것을 고른다
- 그 점수를 ‘모델 성능’이라고 보고한다
- **시험 문제를 보면서 답을 고른 것과 같다**
- 실제 배포하면 그 점수가 안 나온다

## 올바른 절차

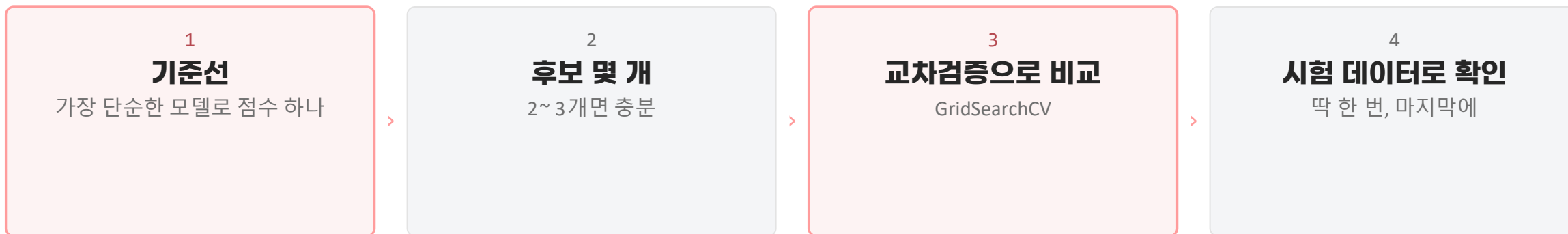
훈련 데이터 안에서 고른다

- ① 훈련 데이터로 교차검증 → 손잡이 선택
- ② 선택한 설정으로 훈련 데이터 전체에 fit
- ③ 시험 데이터는 맨 마지막에 한 번만
- GridSearchCV 는 ①②를 자동으로 해 준다

`gs.fit(X_train, y_train) → gs.score(X_test, y_test)`

지난주 ‘fit 은 train, 평가는 test’ 에 한 줄이 추가 - 고르는 것도 train 안에서

## 모델을 고르는 절차 - 네 단계



- 기준선이 있어야 ‘좋아졌다’를 말할 수 있습니다. 로지스틱 회귀 0.905 가 오늘 데이터의 기준선입니다.
- 후보를 열 개씩 늘리지 말 것. 시간만 들고 결론은 같음. 어떤 트리도 로지스틱을 못 이김
- 기준선을 못 이기면 복잡한 모델을 버리기. 이것이 ‘단순한 모델이 이겼다’의 실무적 결론.



## 오늘 반드시 가져갈 세 가지

### ① 정답이 없어도 구조는 찾는다

KMeans 는 정답 없이 73% 를 갈랐고,  
PCA 는 세 센서가 하나라는 걸 알아냈다.  
단, 그 결과는 사람이 확인해야 라벨이 된다.

### ② 지표는 있지만 결정은 사람이

관성.실루엣은 참고 의견.  $k$  는 목적이 정한다. '정답이 없다'는 말의 진짜 뜻.

### ③ 손잡이는 곡선의 꼭대기에서

하이퍼파라미터는 훈련.시험 곡선이 벌어지기 직전에 멈춘다. 고르는 것도 train 안에서, 시험 데이터는 한 번만.

## (과제) KMeans 와 PCA 를 각각 세 줄로 빈 화면에 재현

- 과제 - 실습 노트북 ①~④ 를 끝까지 실행
- 다음 시간 - 딥러닝 시작. 지난 학기 Keras 의 Sequential → compile → fit 을 기억해 오세요.

**정답이 없어도 구조는 찾는다 · 결정은 사람이 한다**

다음 시간에 봅시다.